

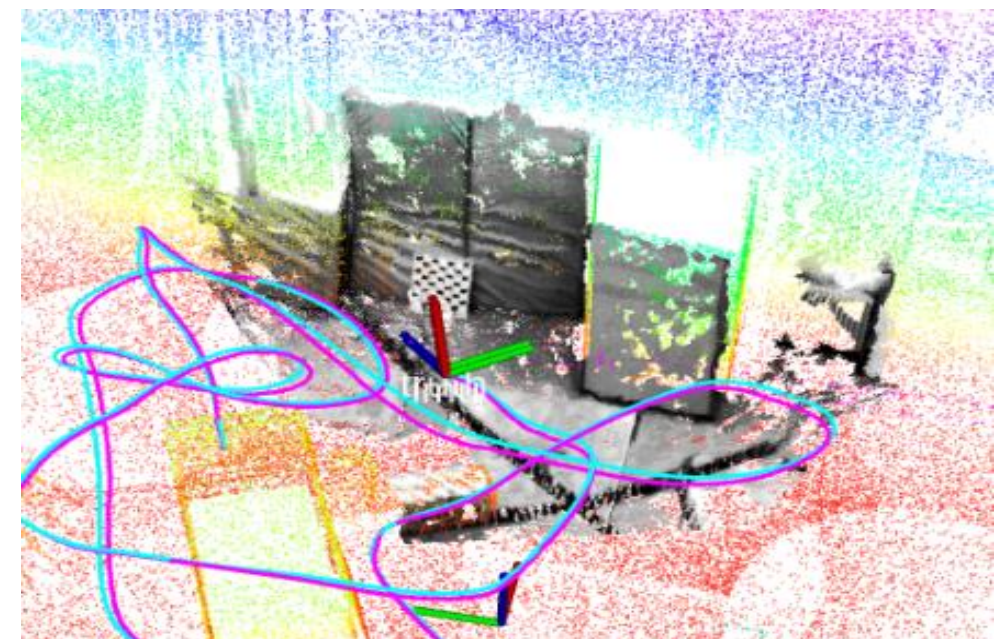
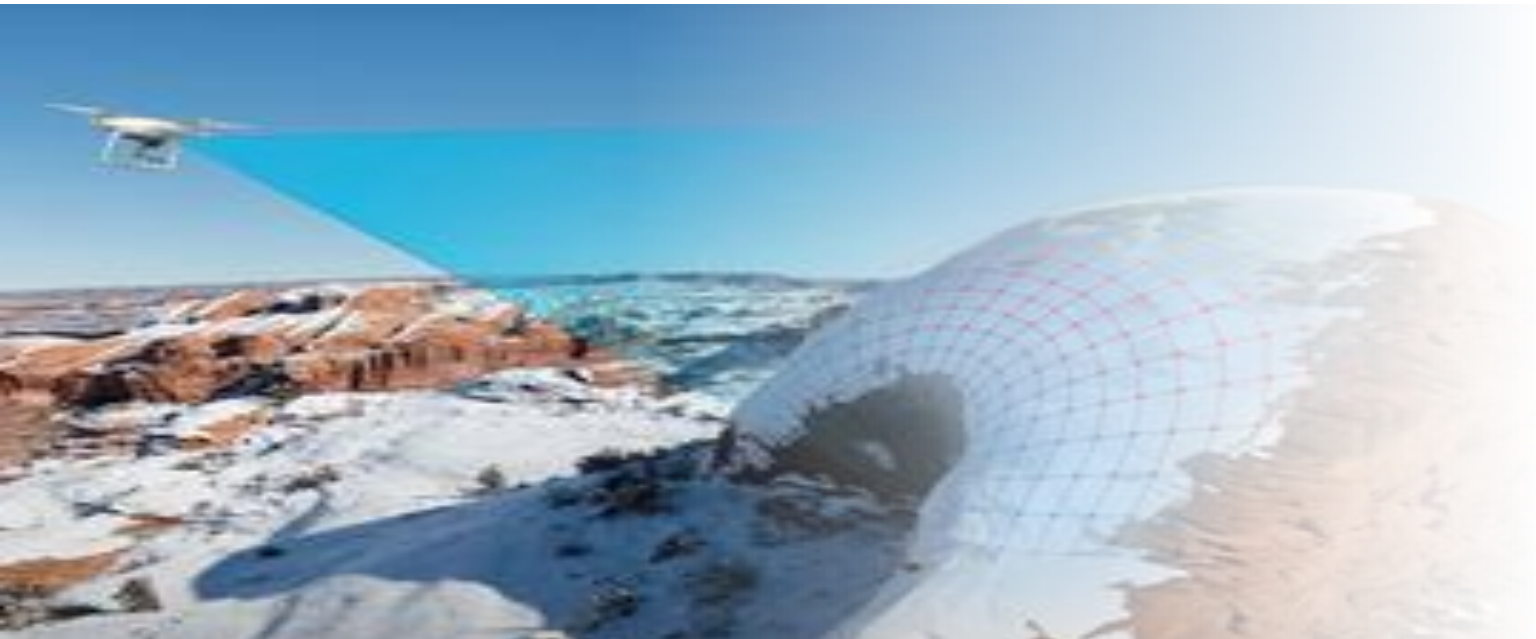
# Robotics & State Estimation (RiSE) Tutorials

## Learning Based SLAM & Dense Mapping

Prof. Xingxing Zuo

Assistant Professor of Robotics

MBZUAI



# Today's Outline

- **Learning-Based SLAM VS Classical SLAM**
- **Where Learning Enters SLAM**
- **Learning-based Features**
- **Descriptor Learning**
- **Learning Depth/Geometry**
- **Optical Flow Learning**
- **Learning-based Differentiable VO**
- **Learning in Optimization**
- **3D Map Representations**
- **Neural Map Representations -- NeRF**
- **Truncated Signed Distance Function (TSDF)**
- **Feed Forward 3D Foundation Models – offline**
- **Feed Forward 3D Foundation Models – online**
- **3D Gaussian Splatting**
- **NeRF vs Gaussian Splatting**
- **SLAM and Dense Mapping in Robotics**

**Classical SLAM works well but has limitations:**

- Handcrafted features
- Sensitivity to environment
- Difficult tuning

**Deep learning offers:**

- Data-driven representations
- Improved robustness

# Where Learning Enters SLAM

## Break SLAM into components:

- Perception (front-end)
- State estimation (back-end)
- Mapping / representation

## 🔑 Learning can be used in:

- Feature detection & description
- Depth/Geometry estimation
- Motion estimation (VO/VIO)
- Place recognition (loop closure)
- Map representation
- Optimization Backend

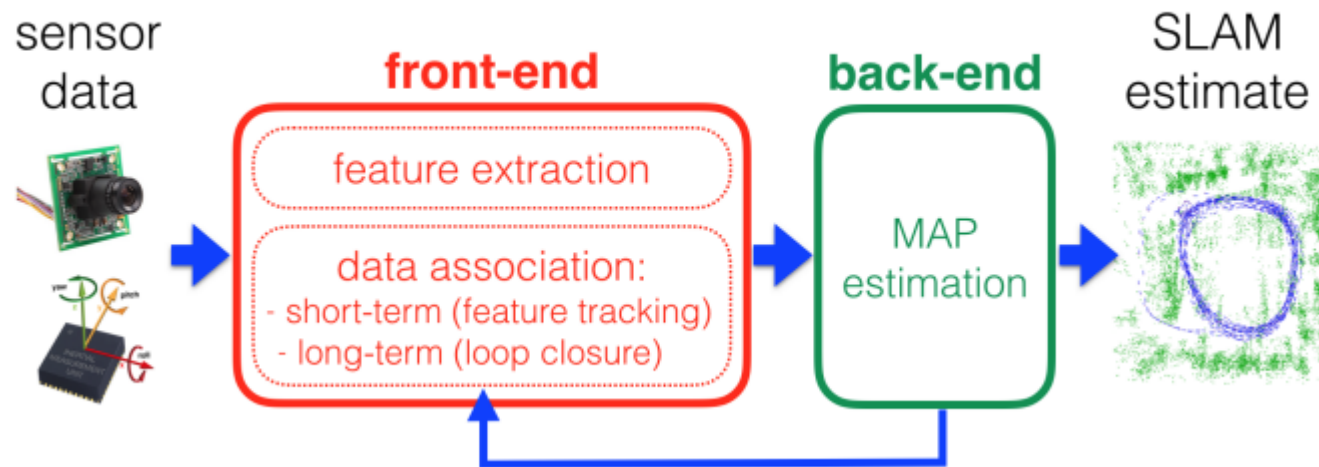


Fig. A typical pipeline of feature-based SLAM [1]

# Learning-based Features

- Learning-based Features:
  - SuperPoint, R2D2
  - Learned keypoints + descriptors
- Advantages:
  - Robust to illumination
  - Better repeatability

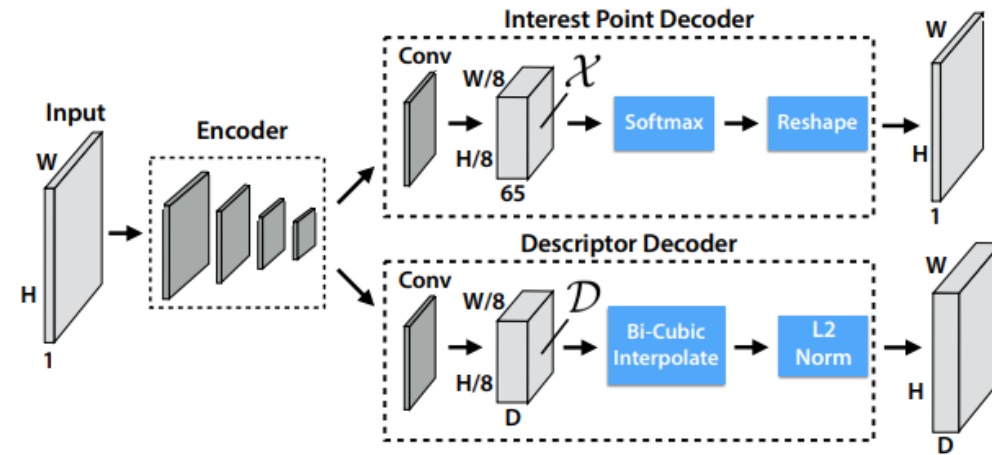


Fig. Superpoint neural network architecture. [1]

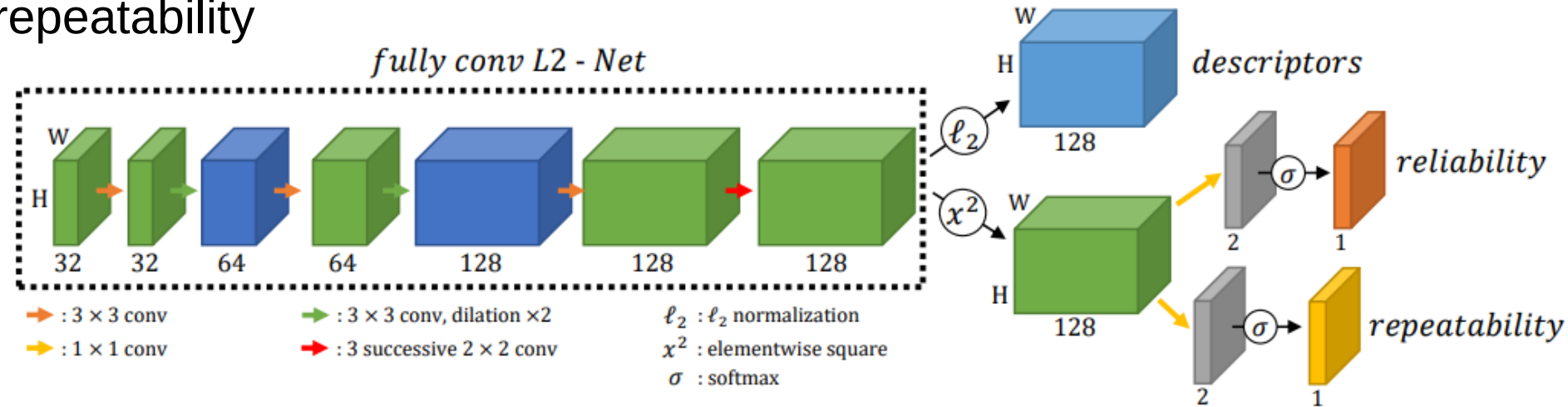


Fig. R2D2 network for jointly learning repeatable and reliable matches. [2]

**Repeatability:** Detect same points across views; ensures consistent matching

**Reliability:** Select features with distinctive descriptors, avoid ambiguous matches

[1] DeTone, D., Malisiewicz, T., & Rabinovich, A.. Superpoint: Self-supervised interest point detection and description. CVPR workshops 2018.

[2] Revaud, J., De Souza, C., Humenberger, M., & Weinzaepfel, P. (2019). R2D2: Reliable and repeatable detector and descriptor. Neurips 2019.

## Key Idea

$$\mathbf{f}_i = \Phi(\mathbf{p}_i), \quad \mathbf{f}_i \in \mathbb{R}^d$$

- Learn a function  $\Phi(\cdot)$  that maps 3D points to descriptors
- Corresponding points  $\rightarrow$  **similar features**
- Non-corresponding points  $\rightarrow$  **dissimilar features**

Example:

- FCGF (3D)
- SuperPoint (2D)

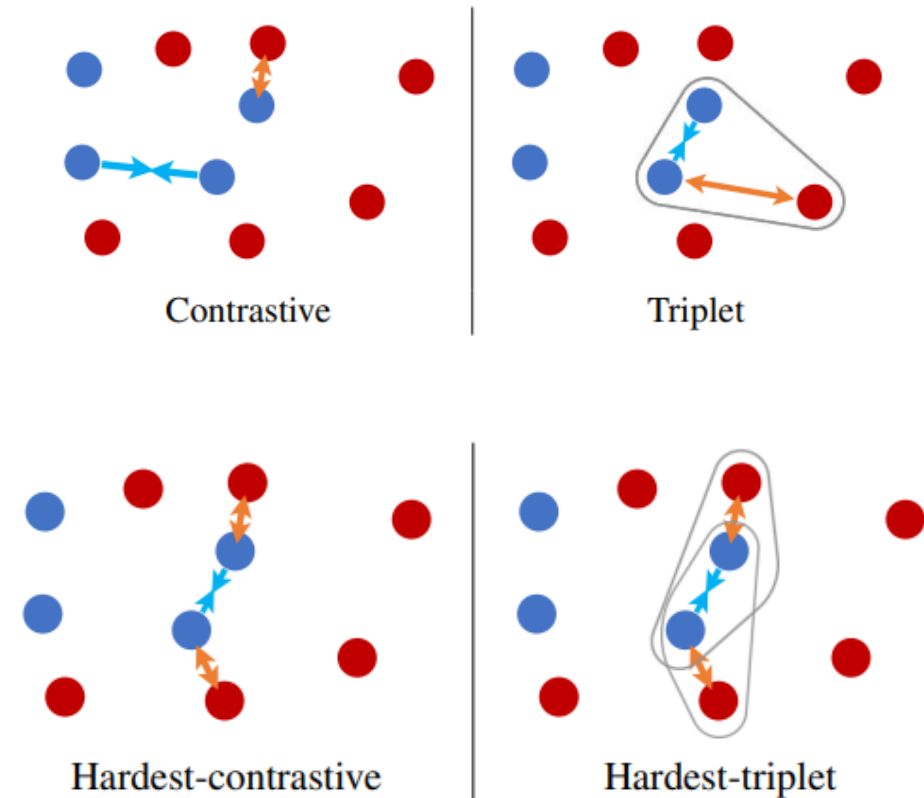


Fig. Sampling and negative-mining strategy for training FCGF. [1]

[1] Choy C, Park J, Koltun V. Fully convolutional geometric features, CVPR 2019: 8958-8966.

[2] Revaud, J., De Souza, C., Humenberger, M., & Weinzaepfel, P. (2019). R2D2: Reliable and repeatable detector and descriptor. Neurips 2019.



# Learning Depth/Geometry

- Monocular depth estimation
- Multi-View Stereo
- Feedforward point cloud prediction

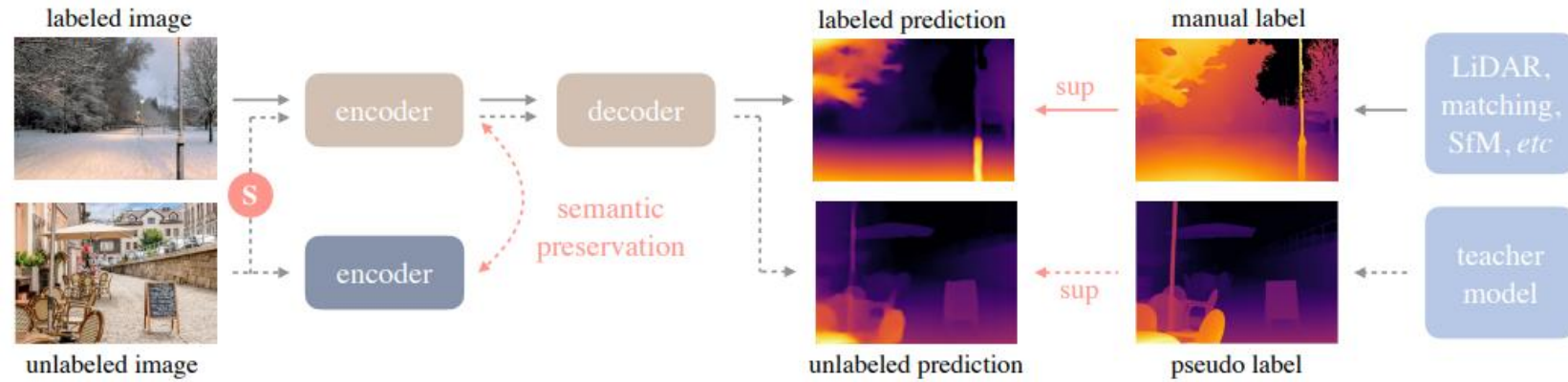


Fig. Pipeline of Depth Anything. [1]

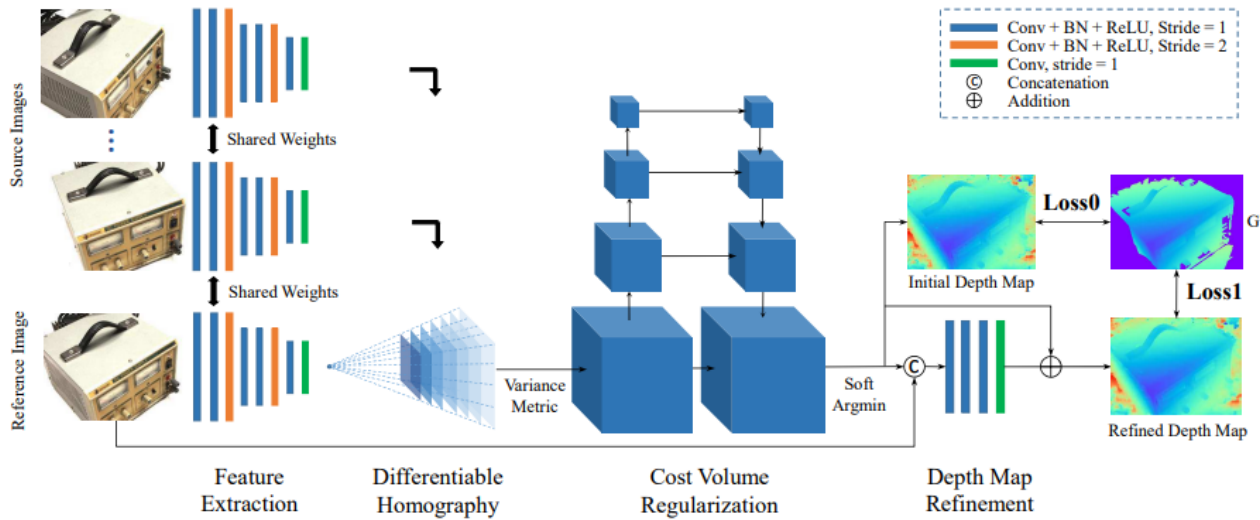


Fig. Pipeline of MVSNet. [2]

[1] Yang, L., Kang, B., Huang, Z., Xu, X., Feng, J., & Zhao, H. (2024). Depth anything: Unleashing the power of large-scale unlabeled data. CVPR 2024.

[2] Yao Y, Luo Z, Li S, et al. Mvsnet: Depth inference for unstructured multi-view stereo, ECCV 2018.

[3] Lin H, Chen S, Liew J, et al. Depth anything 3: Recovering the visual space from any views. arXiv preprint arXiv:2511.10647, 2025.

# Learning Depth/Geometry

- Monocular depth estimation
- Multi-View Stereo
- Feedforward point cloud prediction

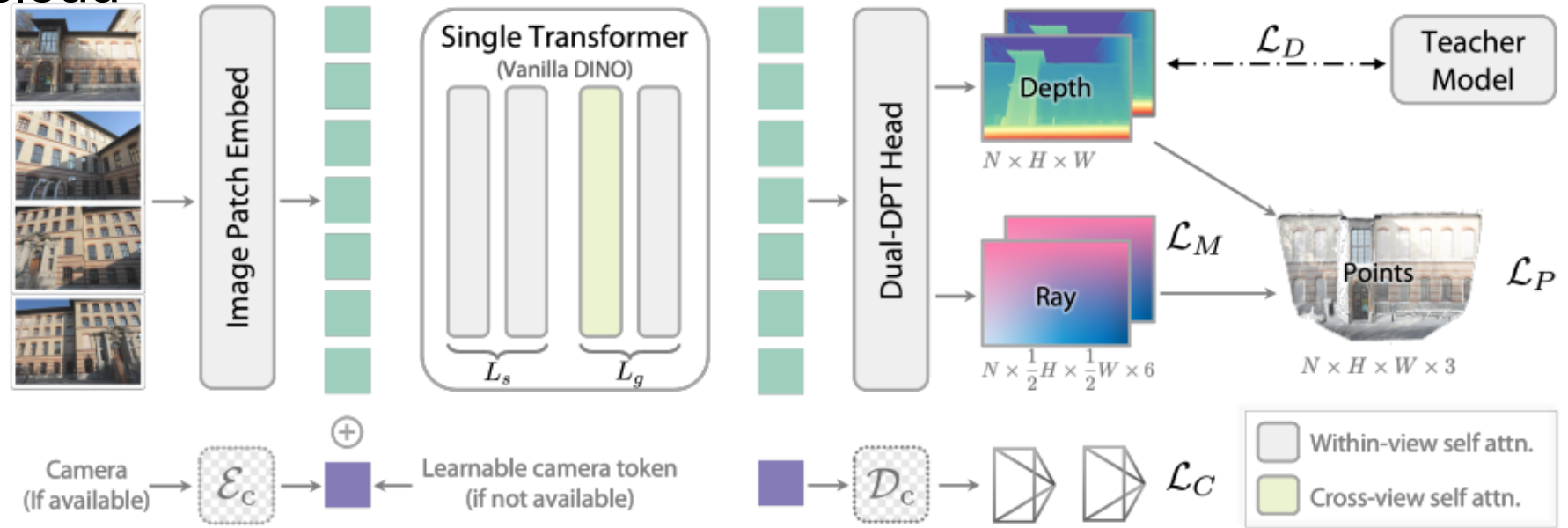


Fig. Pipeline of Depth Anything 3. [3]

[1] Yang, L., Kang, B., Huang, Z., Xu, X., Feng, J., & Zhao, H. (2024). Depth anything: Unleashing the power of large-scale unlabeled data. CVPR 2024.

[2] Yao Y, Luo Z, Li S, et al. Mvsnet: Depth inference for unstructured multi-view stereo, ECCV 2018.

[3] Lin H, Chen S, Liew J, et al. Depth anything 3: Recovering the visual space from any views. arXiv preprint arXiv:2511.10647, 2025.



- RAFT: Dense correspondence estimation

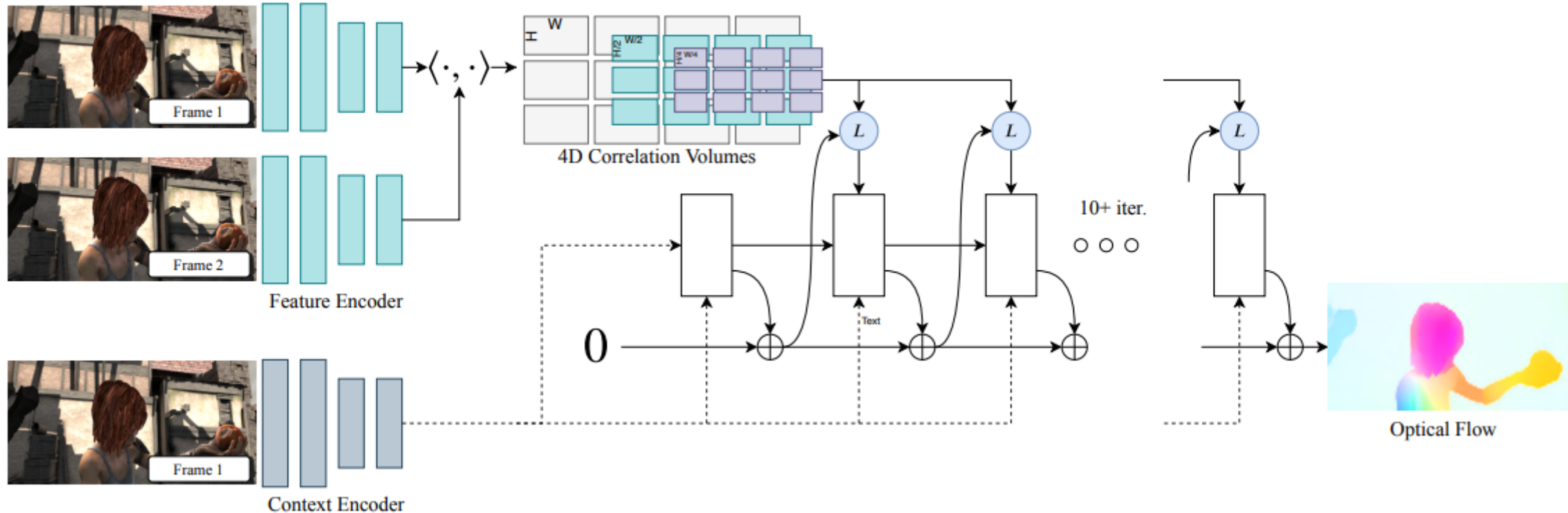


Fig. RAFT [1] consists of 3 main components: (1) A feature encoder that extracts features. (2) A correlation layer which constructs a 4D  $W \times H \times W \times H$  correlation volume by taking the inner product of all pairs of feature vectors. The last 2-dimensions of the 4D volume are pooled at multiple scales to construct a set of multi-scale volumes. (3) An update operator which recurrently updates optical flow by using the current estimate to look up values from the set of correlation volumes.

# Learning-based Differentiable VO

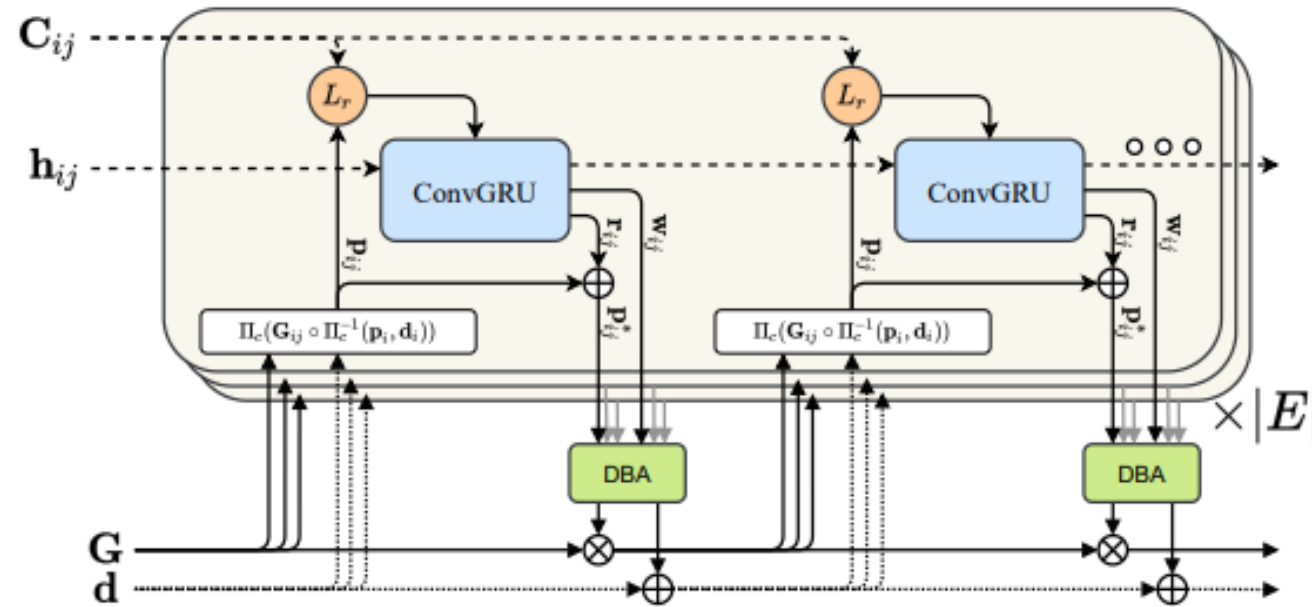
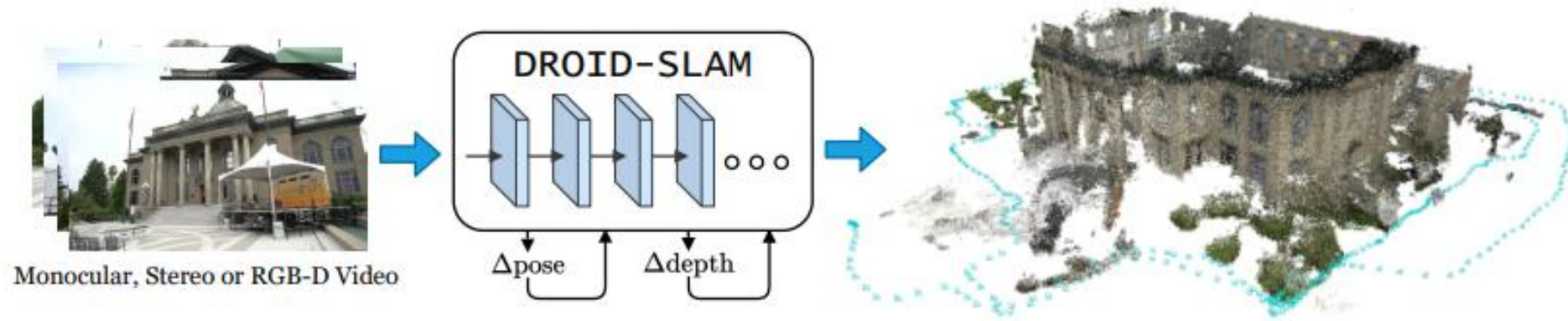


Fig. Droid SLAM [1] overview and its differentiable bundle adjustment update operator.

# Learning in Optimization

- Learned cost functions
- Learned uncertainty
- Neural bundle adjustment

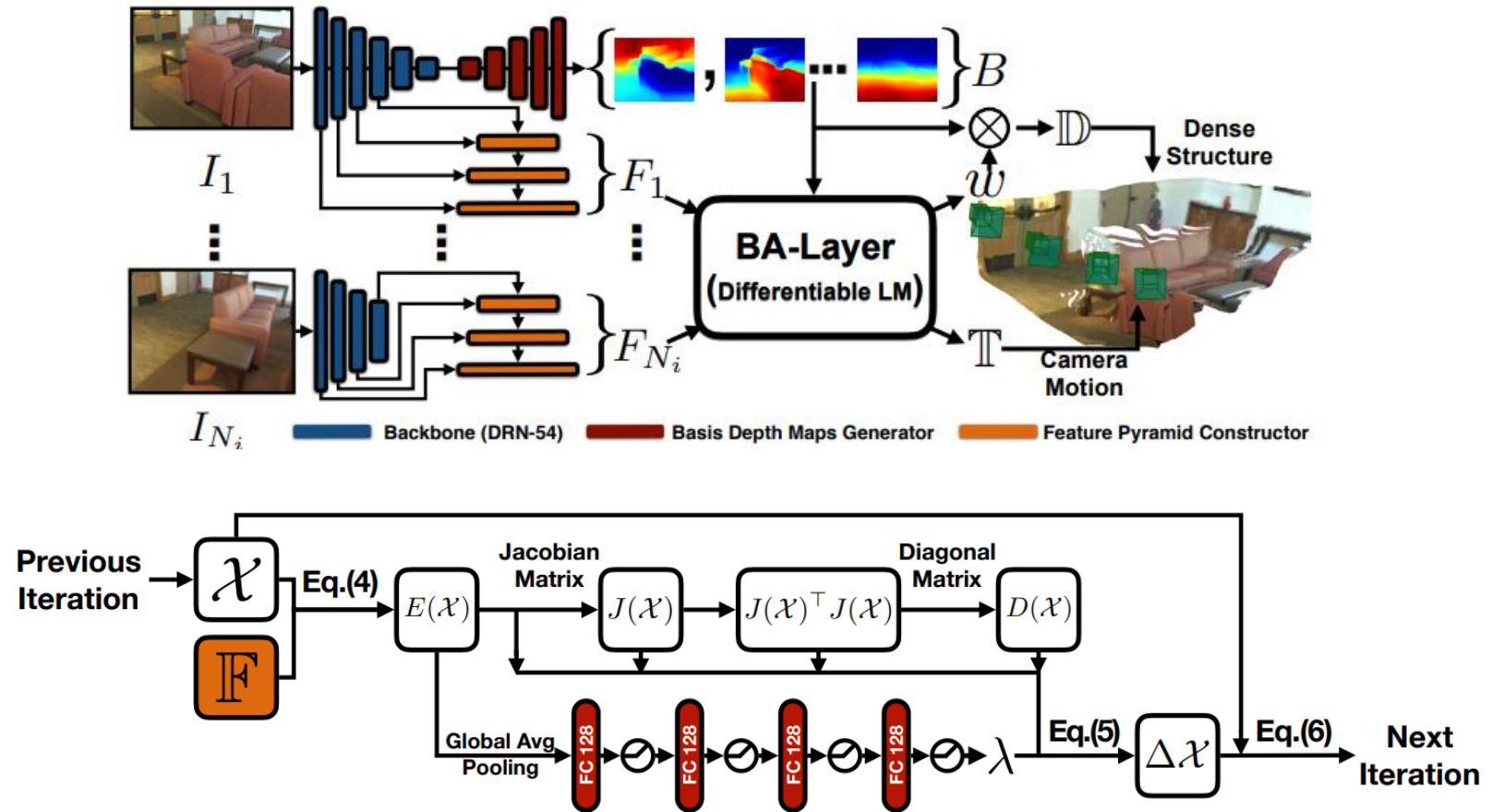
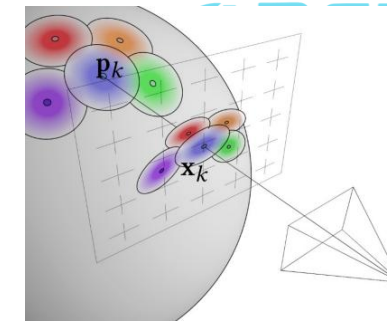
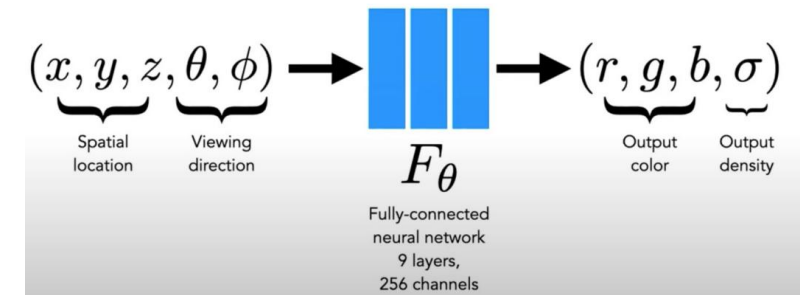


Fig. Overview of BA-Net structure [1]. A Basis Depth Maps Generator that generates a set of basis depth maps, a Feature Pyramid Constructor that constructs multi-scale feature maps, and a BA-Layer that optimizes both the depth map and the camera poses through a novel differentiable LM algorithm with damping parameter  $\lambda$  predicted.

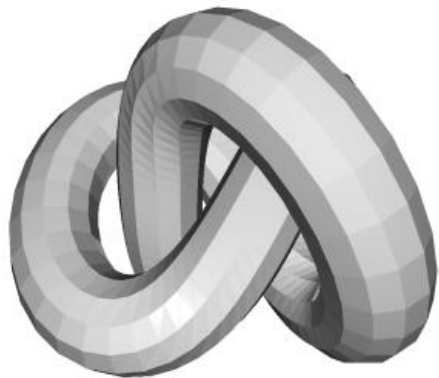
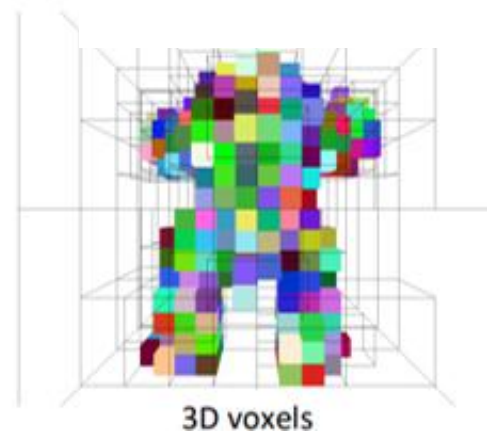
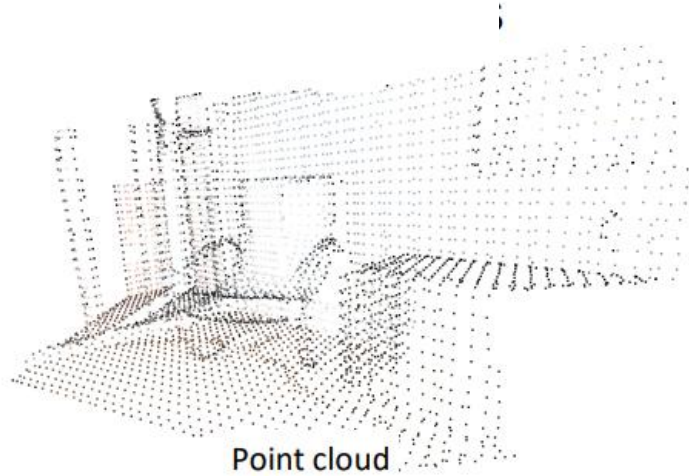
# 3D Map Representations



3D Gaussians



Neural Radiance Fields

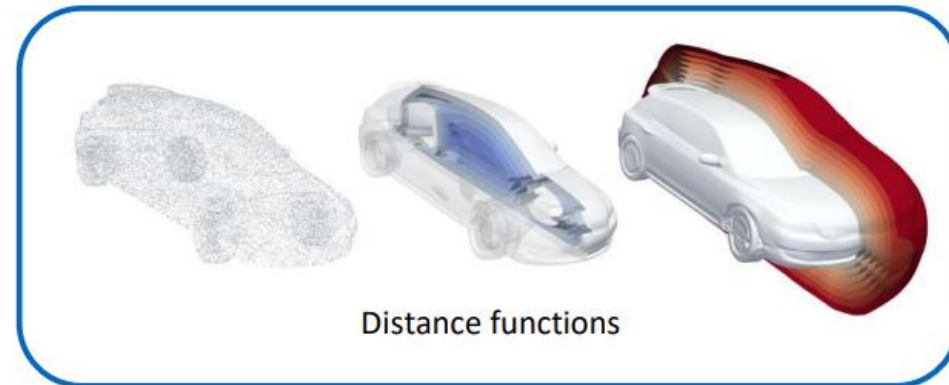


Meshes



An **image** that represents how far each pixel  $\mathbf{p}$  is  $D[\mathbf{p}] \in \mathbb{R}^+$

Depth images

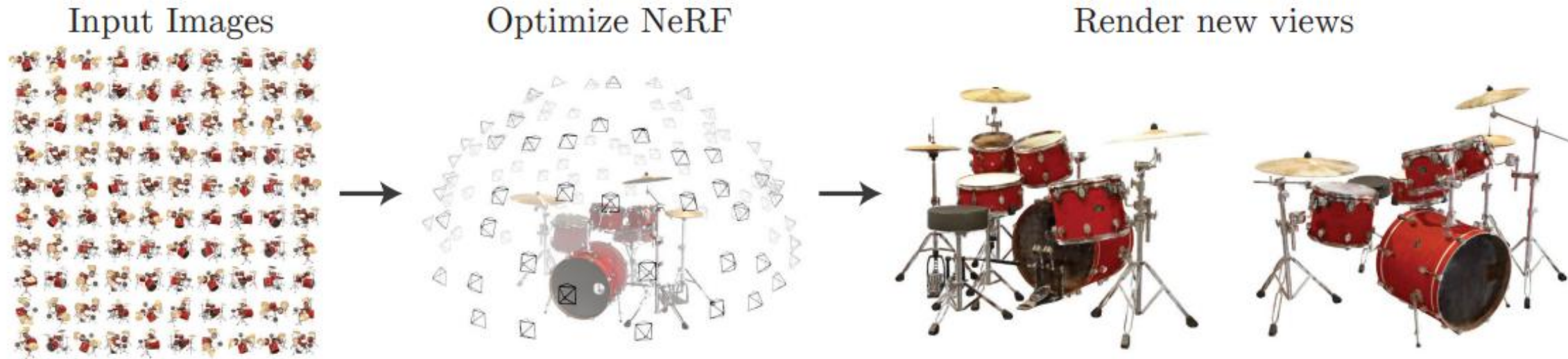




# Neural Map Representations

## NeRF (Neural Radiance Fields)

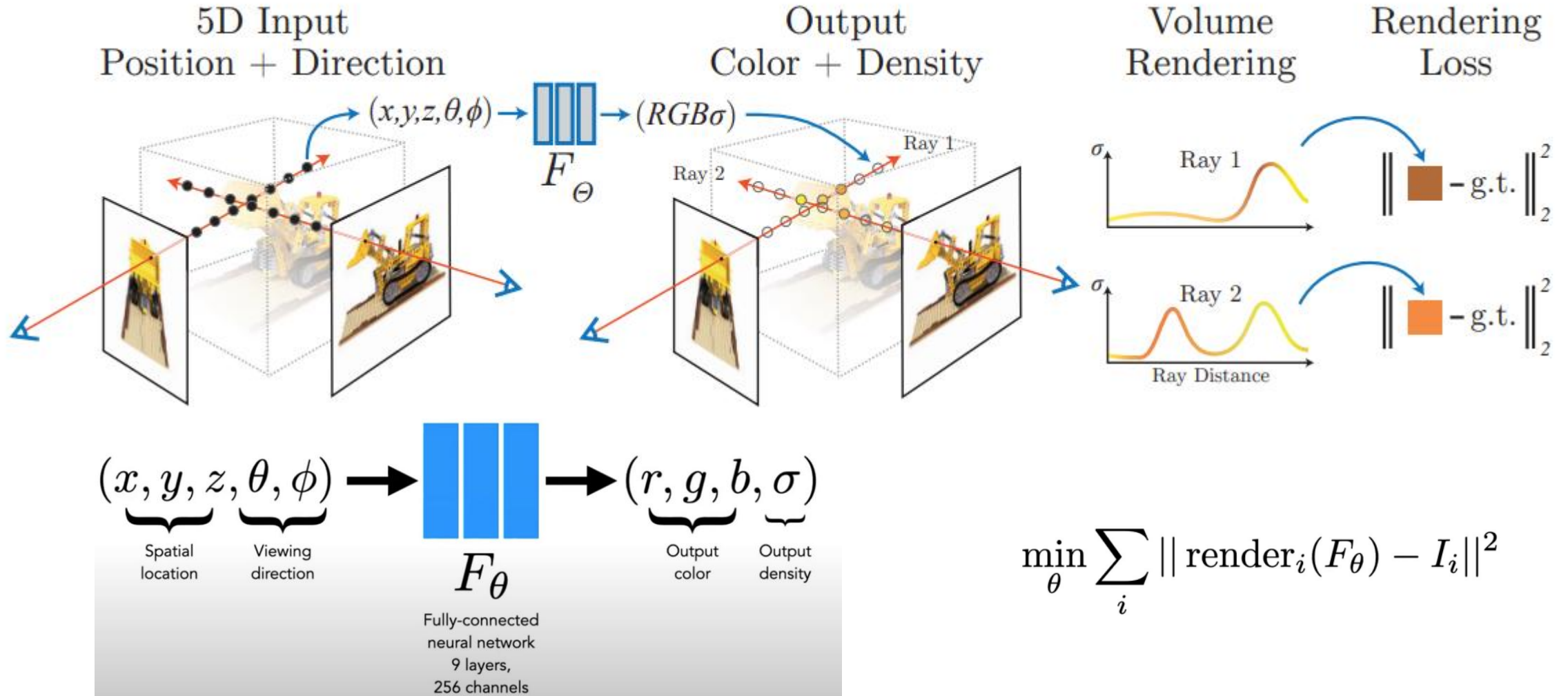
It is a scene representation using a fully-connected (non-convolutional) deep network, whose input is a single continuous 5D coordinate (spatial location  $(x, y, z)$  and viewing direction  $(\theta, \phi)$ ) and whose output is the volume density and view-dependent emitted radiance at that spatial location. It is excellent for novel view synthesis.





# Neural Map Representations

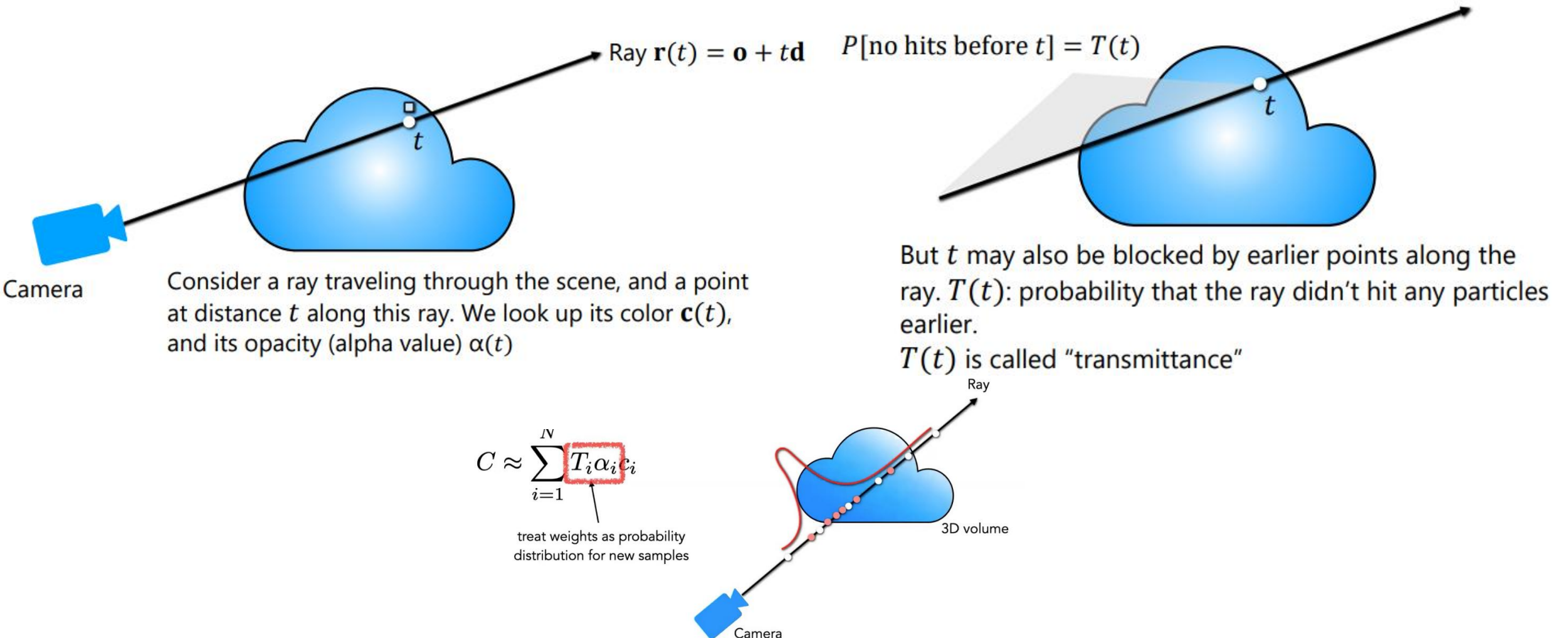
## NeRF (Neural Radiance Fields)



# Neural Map Representations

## NeRF (Neural Radiance Fields)

### □ Volumetric formulation



# Neural Map Representations

## NeRF (Neural Radiance Fields)

### □ Volumetric formulation

Rendering model for ray  $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ :

$$\mathbf{c} \approx \sum_{i=1}^n T_i \alpha_i \mathbf{c}_i$$

final rendered color along ray      weights      colors

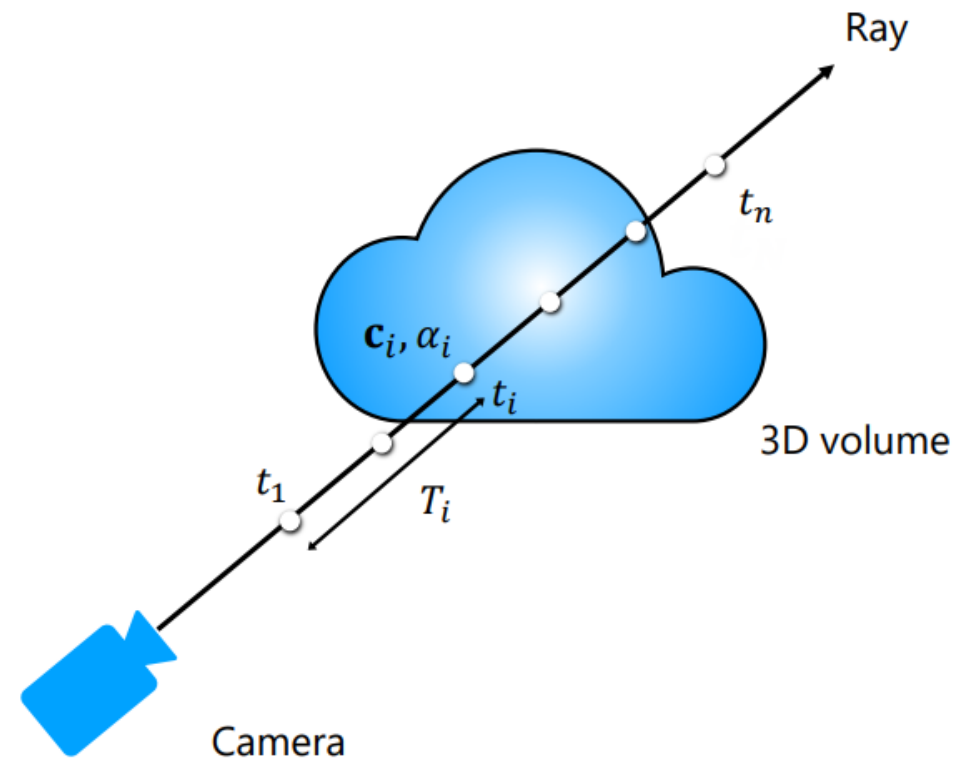
How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Computing the color for a set of rays through the pixels of an image yields a rendered image



$\alpha$  (Opacity--Discrete) is not directly stored in the volume, but instead is derived from a stored **volume density** sigma



$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$

$\delta_i$  distance between samples along a ray

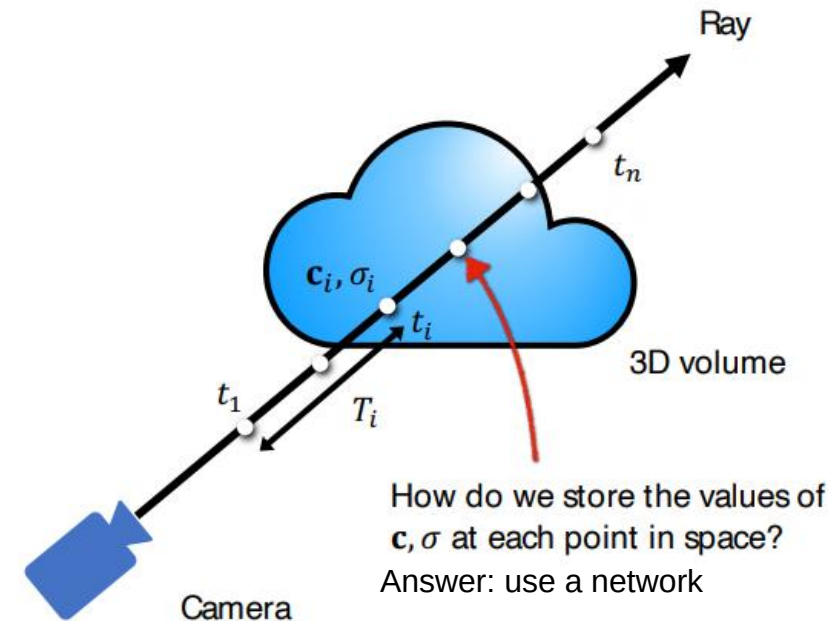
# Neural Map Representations

## NeRF (Neural Radiance Fields)

### □ Volumetric formulation

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i, \text{ where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right),$$

where  $\delta_i = t_{i+1} - t_i$  is the distance between adjacent samples. This function for calculating  $\hat{C}(\mathbf{r})$  from the set of  $(\mathbf{c}_i, \sigma_i)$  values is trivially differentiable and reduces to traditional alpha compositing with alpha values  $\alpha_i = 1 - \exp(-\sigma_i \delta_i)$ .



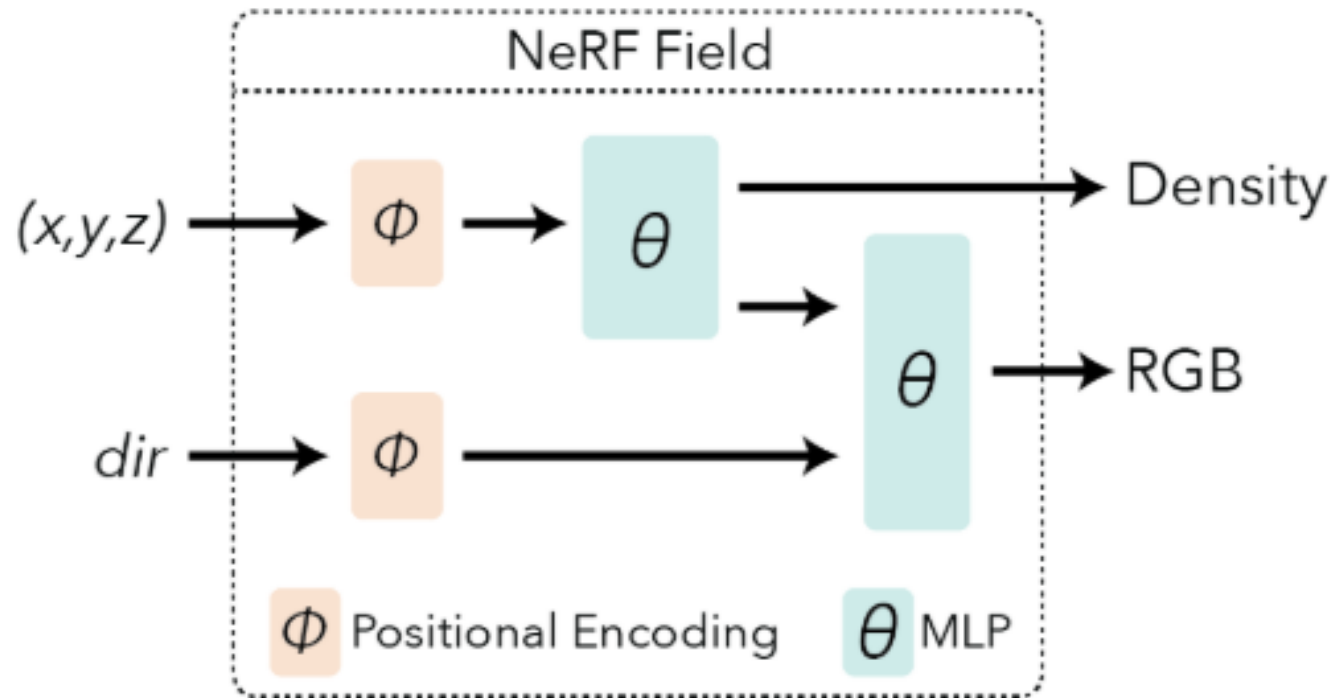
# Neural Map Representations

## NeRF (Neural Radiance Fields)

### Neural Network Architecture

Positional Embedding for variable  $p$ :

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p)) .$$

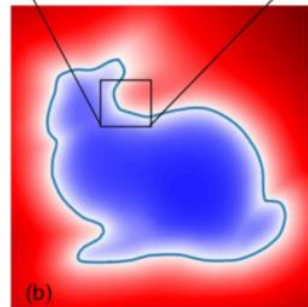
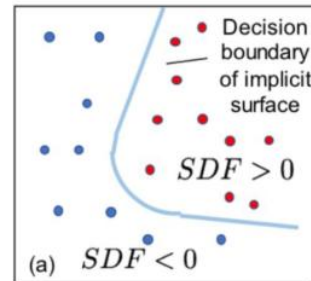


NeRF network architecture: MLP  $\theta$  first processes the input 3D coordinate  $x$  with 8 fully-connected layers, and outputs  $\sigma$  and a 256-dimensional feature vector. This feature vector is then concatenated with the camera ray's viewing direction and passed to one additional fully-connected layer that output the view-dependent RGB color.



**Definition 1.** The *signed distance function* (SDF) of a set  $\mathcal{O} \subset \mathbb{R}^n$  is a function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  that measures the signed distance from a point  $p \in \mathbb{R}^n$  to the set boundary  $\partial\mathcal{O}$ , defined as:

$$f_{\text{SDF}}(p; \mathcal{O}) \triangleq \begin{cases} \min_{y \in \partial\mathcal{O}} \|p - y\|_2, & p \notin \mathcal{O}, \\ -\min_{y \in \partial\mathcal{O}} \|p - y\|_2, & p \in \mathcal{O}. \end{cases} \quad (1)$$

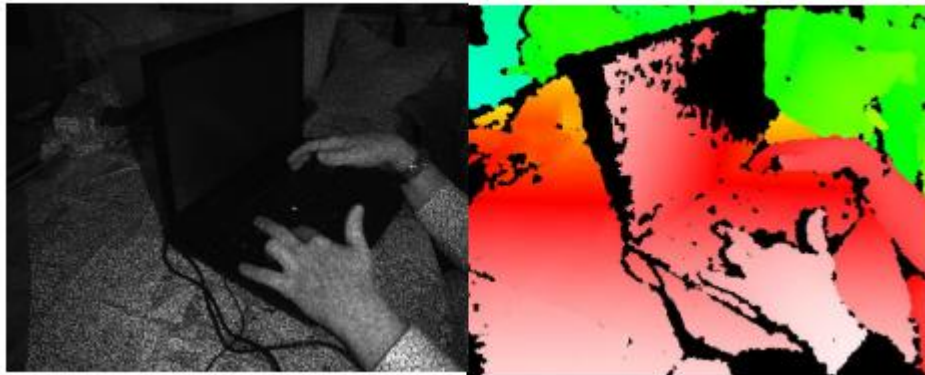


Park et al. 2019

# Truncated Signed Distance Function (TSDF)

## Case Study: KinectFusion

Context: Kinect RGB-D camera released by Microsoft in 2010.



### KinectFusion: Real-Time Dense Surface Mapping and Tracking\*

Richard A. Newcombe  
Imperial College London

Shahram Izadi  
Microsoft Research

Otmar Hilliges  
Microsoft Research

David Molyneaux  
Microsoft Research  
Lancaster University

David Kim  
Microsoft Research  
Newcastle University

Andrew J. Davison  
Imperial College London

Pushmeet Kohli  
Microsoft Research

Jamie Shotton  
Microsoft Research

Steve Hodges  
Microsoft Research

Andrew Fitzgibbon  
Microsoft Research

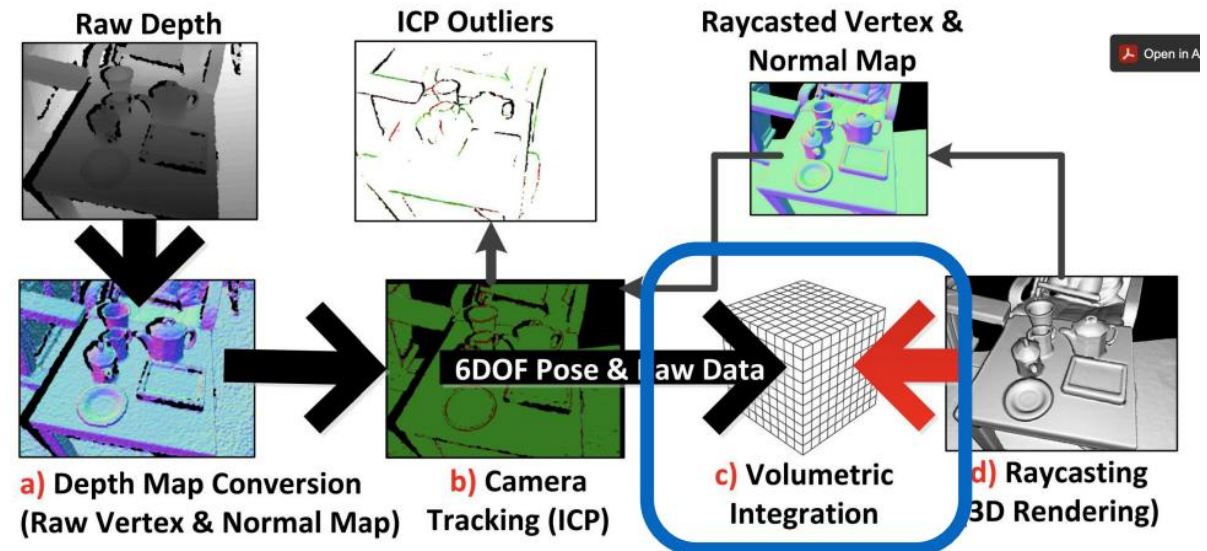
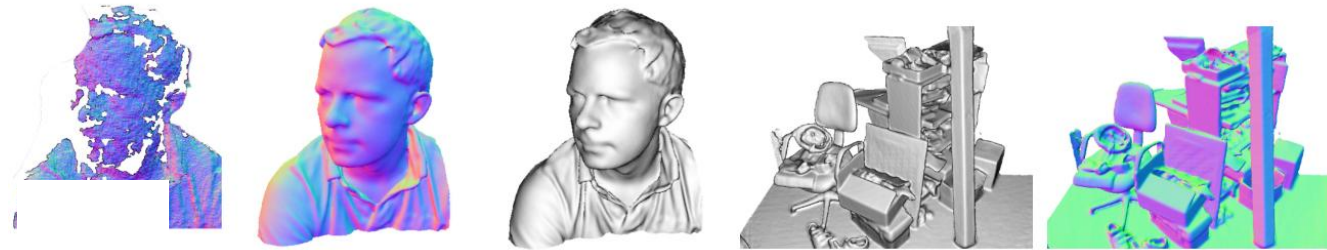
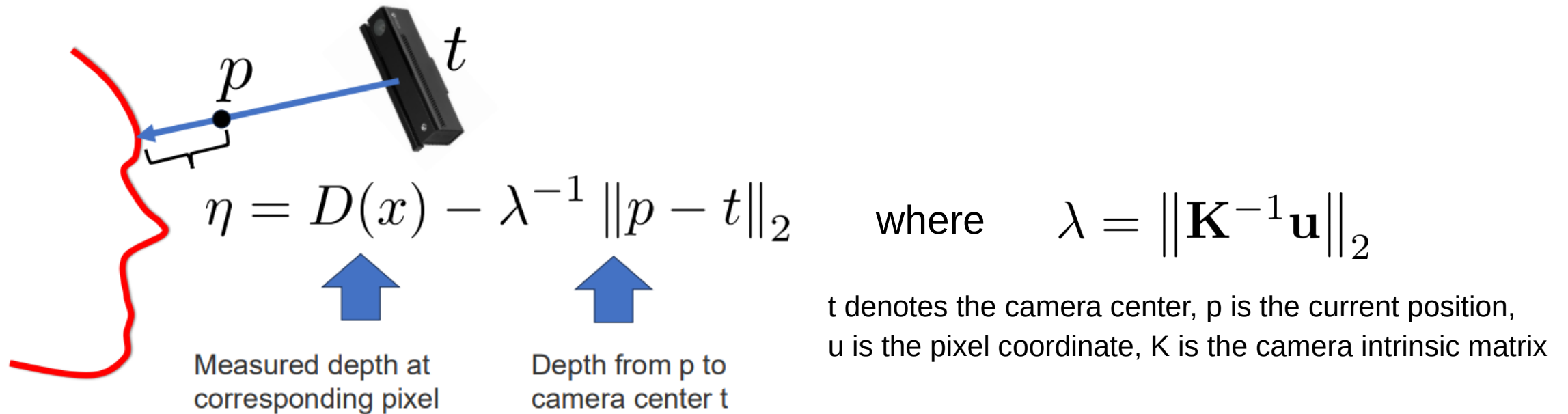


Image credit: Newcombe

Demo of KinectFusion: <https://www.youtube.com/watch?v=KOUSSIKUJ-A>

# Truncated Signed Distance Function (TSDF)

- KinectFusion uses truncated SDF (TSDF) computed by projective distance



# Truncated Signed Distance Function (TSDF)

- KinectFusion uses truncated SDF (TSDF) computed by projective distance

|      |      |      |      |     |     |     |   |   |
|------|------|------|------|-----|-----|-----|---|---|
| -0.9 | -0.3 | 0.0  | 0.2  | 1   | 1   | 1   | 1 | 1 |
| -1   | -0.9 | -0.2 | 0.0  | 0.2 | 1   | 1   | 1 | 1 |
| -1   | -0.9 | -0.3 | 0.0  | 0.1 | 0.9 | 1   | 1 | 1 |
| -1   | -0.8 | -0.3 | 0.0  | 0.2 | 0.8 | 1   | 1 | 1 |
| -1   | -0.9 | -0.4 | -0.1 | 0.1 | 0.8 | 0.9 | 1 | 1 |
| -1   | -0.7 | -0.3 | 0.0  | 0.3 | 0.6 | 1   | 1 | 1 |
| -1   | -0.7 | -0.4 | 0.0  | 0.2 | 0.7 | 0.8 | 1 | 1 |
| -0.9 | -0.7 | -0.2 | 0.0  | 0.2 | 0.8 | 0.9 | 1 | 1 |
| -0.1 | -0.0 | 0.0  | 0.1  | 0.3 | 1   | 1   | 1 | 1 |
| 0.5  | 0.3  | 0.2  | 0.4  | 0.8 | 1   | 1   | 1 | 1 |

Image credit: Tim Chen

From every raw depth image, we obtain a normalized and weighted TSDF:

$$F_{R_k}(p) = \begin{cases} \eta/tr, & \text{if } |\eta| \leq tr, \\ \text{sign}(\eta), & \text{otherwise.} \end{cases}$$

$$W_{R_k}(p) \propto \cos(\theta)/D(x)$$

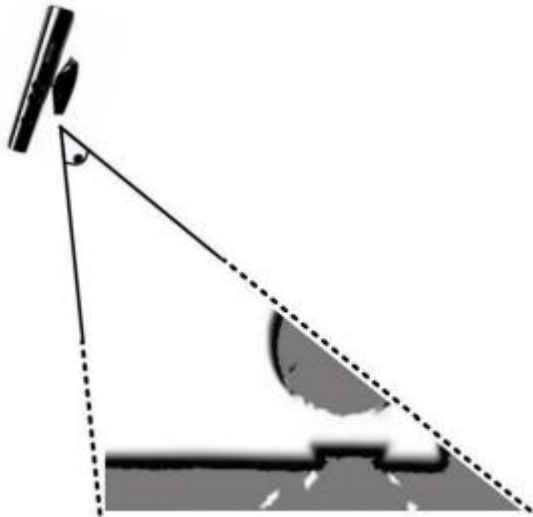
Weight is higher if closer and viewed from a perpendicular viewpoint.



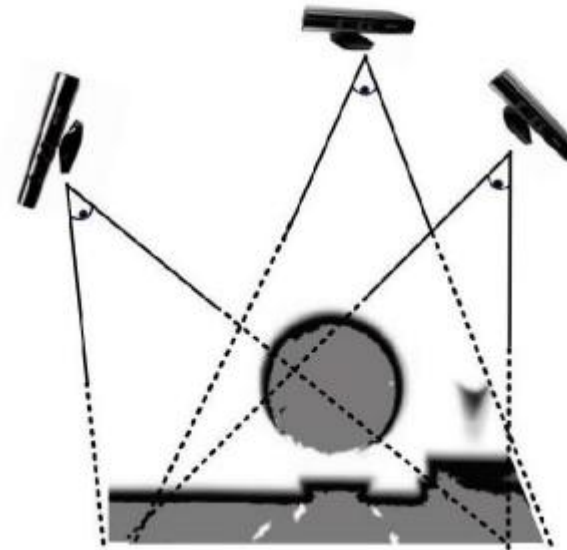
# Truncated Signed Distance Function (TSDF)

## Integration of observations from multi views

Each depth image provides a **partial (noisy)** observation of TSDF.



**Volume integration** recursively update the map given new observation.





# Truncated Signed Distance Function (TSDF)

## Integration of observations from multi views

$$\begin{array}{lcl} \text{Updated TSDF} & & \text{Old TSDF} \quad \text{New Observation} \\ \downarrow & & \downarrow \quad \downarrow \\ F_k(\mathbf{p}) & = & \frac{W_{k-1}(\mathbf{p})F_{k-1}(\mathbf{p}) + W_{R_k}(\mathbf{p})F_{R_k}(\mathbf{p})}{W_{k-1}(\mathbf{p}) + W_{R_k}(\mathbf{p})} \\ \\ W_k(\mathbf{p}) & = & W_{k-1}(\mathbf{p}) + W_{R_k}(\mathbf{p}) \\ \uparrow & & \\ \text{Weight} & & \end{array}$$

Running weighted average implemented on GPU  
(65 gigavoxels/sec, OR,  $\approx 2\text{ms}$  per full volume update for a  $512^3$  voxel reconstruction)

### Why GPU parallelization is feasible for Kinect fusion?

Depth map processing: each pixel can be filtered or back-projected independently.

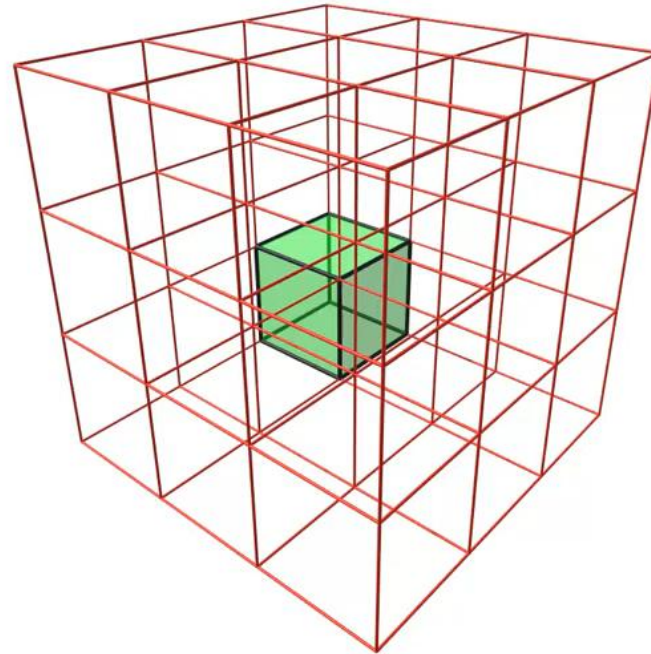
ICP tracking: correspondences and residuals are computed for many pixels in parallel.

Raycasting: each output pixel casts one ray independently through the TSDF volume.

TSDF fusion: each voxel update can be computed independently from projected measurements.

# Marching Cube: 3D Surface from Occupancy/SDF

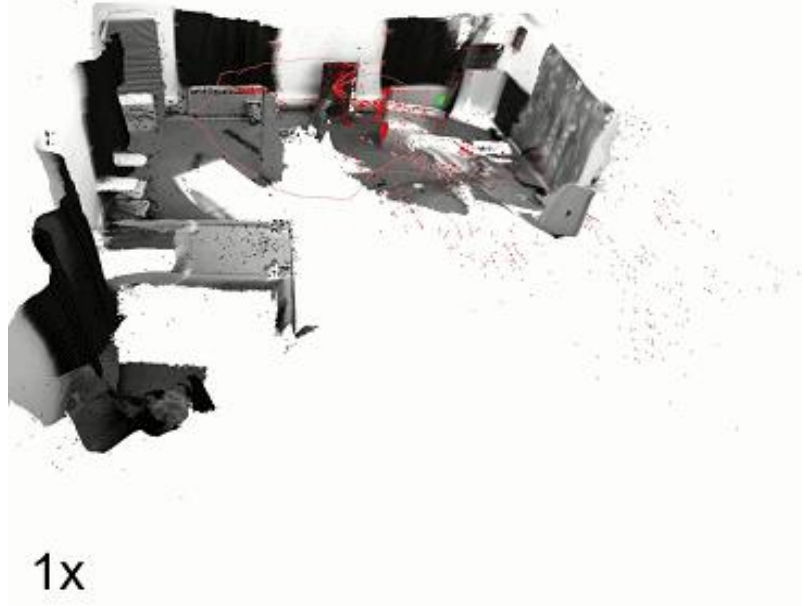
Marching cubes



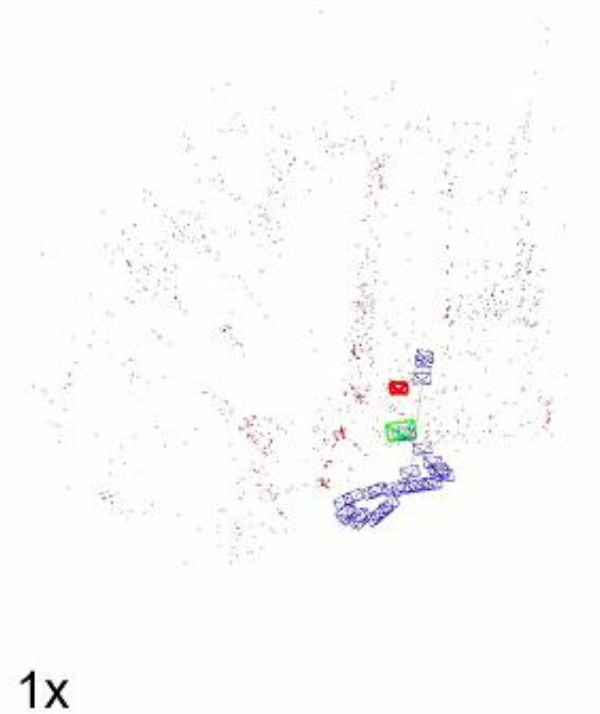
Video Credits: [https://www.youtube.com/watch?v=B\\_xk71YopsA](https://www.youtube.com/watch?v=B_xk71YopsA)

# Case Study: SimpleMapping

EuRoC/V2\_01\_easy



Own dataset/Room\_1



- An incremental visual-inertial dense mapping method that couples MVS network with a sparse VIO
- Dense mapping with great robustness, high efficiency, and remarkable generalization capabilities

# Case Study: SimpleMapping

## AR Demo



Original Reconstructed Scene



Virtual Objects in Reconstructed Scene

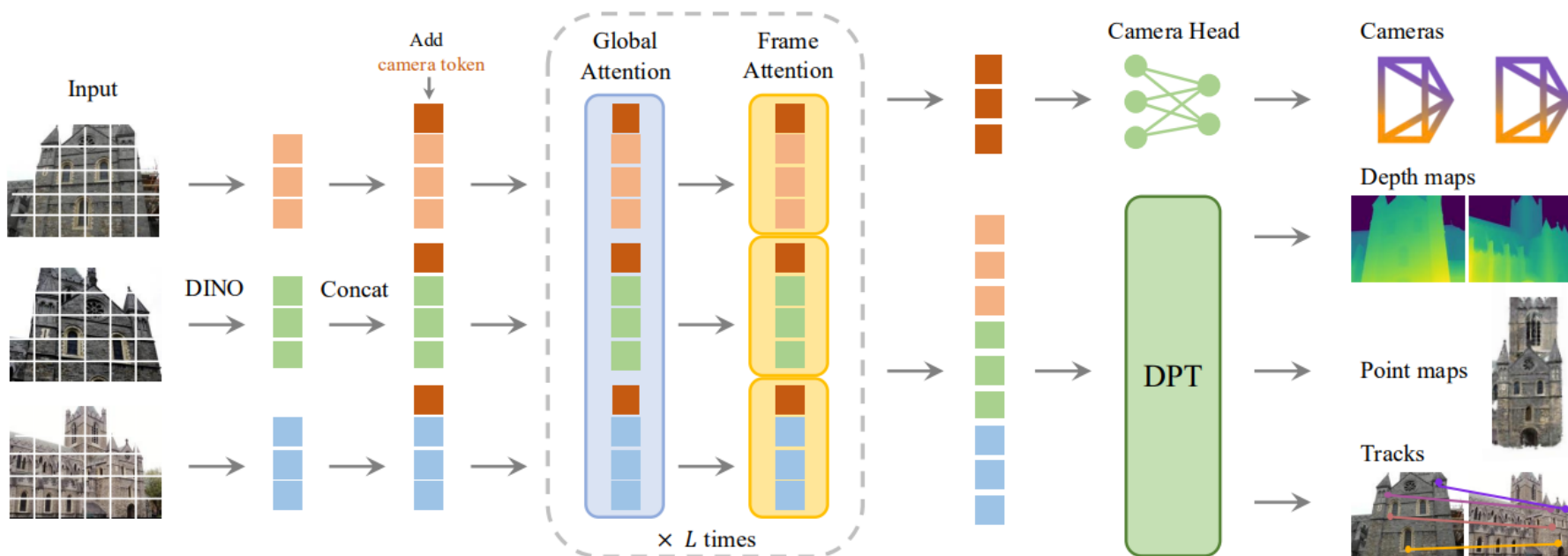
- SimpleMapping allows accurate representation of spatial relationships between real and virtual elements.

- An incremental visual-inertial dense mapping method that couples MVS network with a sparse VIO
- Dense mapping with great robustness, high efficiency, and remarkable generalization capabilities



# Feed Forward 3D Foundation Models -- VGGT

## Offline method



The input is a sequence  $(I_i)_{i=1}^N$  of  $N$  RGB images  $I_i \in \mathbb{R}^{3 \times H \times W}$ , observing the same 3D scene. VGGT's transformer is a function that maps this sequence to a corresponding set of 3D annotations, one per frame:

$$f((I_i)_{i=1}^N) = (\mathbf{g}_i, D_i, P_i, T_i)_{i=1}^N. \quad (1)$$

The transformer thus maps each image  $I_i$  to its camera parameters  $\mathbf{g}_i \in \mathbb{R}^9$  (intrinsic and extrinsic), its depth map  $D_i \in \mathbb{R}^{H \times W}$ , its point map  $P_i \in \mathbb{R}^{3 \times H \times W}$ , and a grid  $T_i \in \mathbb{R}^{C \times H \times W}$  of  $C$ -dimensional features for point tracking. We explain next how these are defined.

$$\mathcal{L} = \mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}} + \lambda \mathcal{L}_{\text{track}}.$$

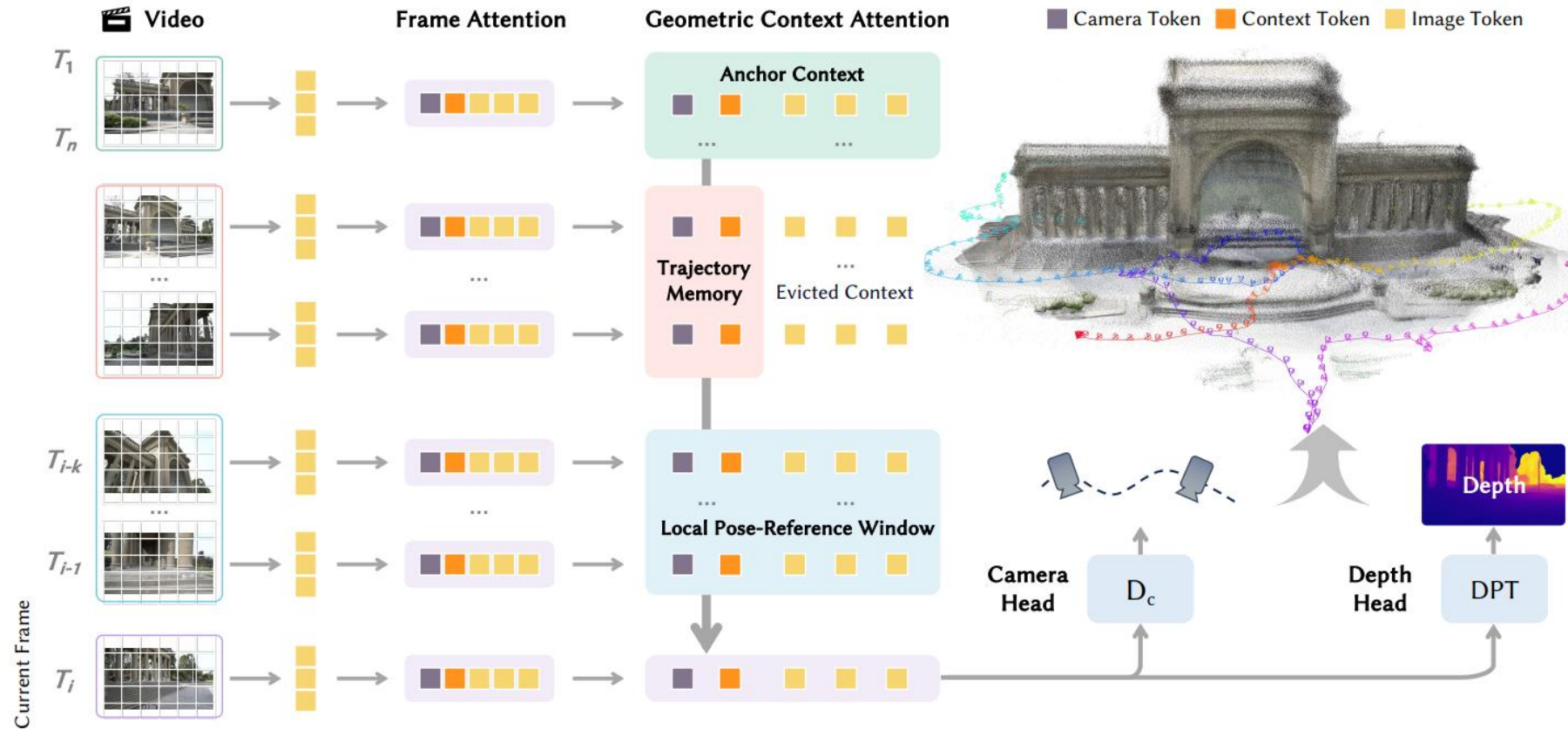
Figure 2. **Architecture Overview.** Our model first patchifies the input images into tokens by DINO, and appends camera tokens for camera prediction. It then alternates between frame-wise and global self attention layers. A camera head makes the final prediction for camera extrinsics and intrinsics, and a DPT [87] head for any dense output.

| Input Frames | 1    | 2    | 4    | 8    | 10   | 20   | 50    | 100   | 200   |
|--------------|------|------|------|------|------|------|-------|-------|-------|
| Time (s)     | 0.04 | 0.05 | 0.07 | 0.11 | 0.14 | 0.31 | 1.04  | 3.12  | 8.75  |
| Mem. (GB)    | 1.88 | 2.07 | 2.45 | 3.23 | 3.63 | 5.58 | 11.41 | 21.15 | 40.63 |

Table 9. **Runtime and peak GPU memory usage across different numbers of input frames.** Runtime is measured in seconds, and GPU memory usage is reported in gigabytes.

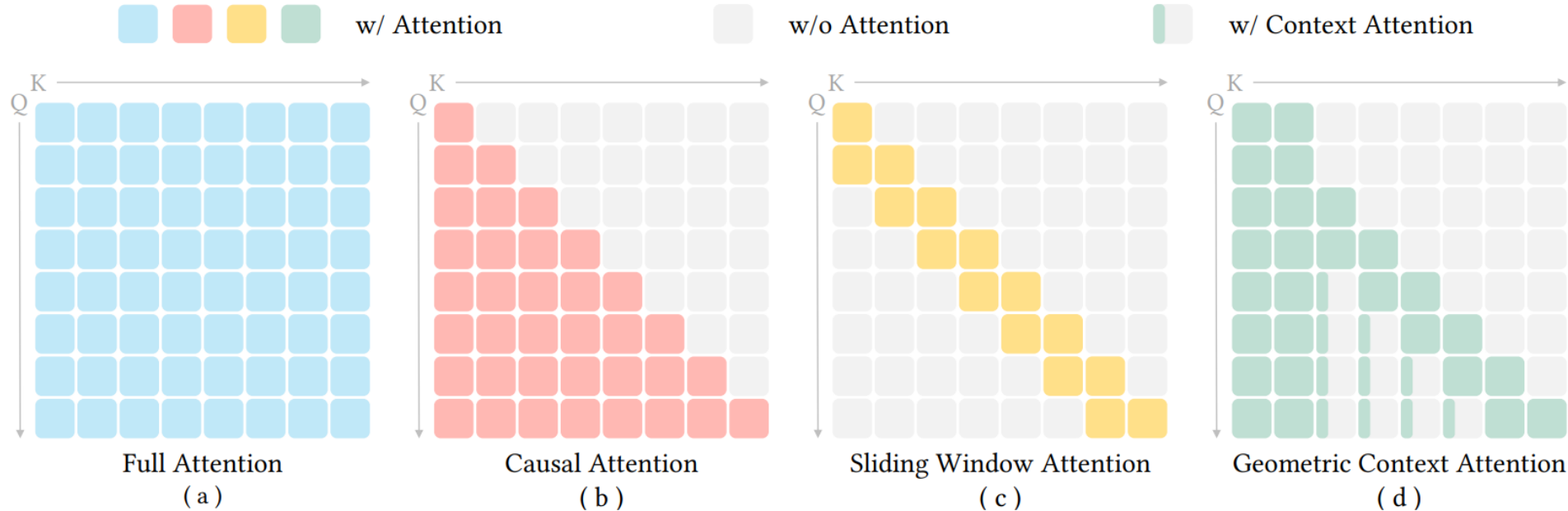
# Feed Forward 3D Foundation Models -- LingBot Map

## Streaming online method



The central challenge of **streaming 3D reconstruction** is managing geometric context: the model must retain sufficient long-range context to ensure global consistency, yet keep its streaming state compact enough for efficient inference.

# Feed Forward 3D Foundation Models -- LingBot Map



## Geometric Context Attention (GCA)

**Goal:** Achieve globally consistent streaming 3D reconstruction with bounded per-frame computation.

- **Anchor context:** First  $n$  frames establish a fixed coordinate system and canonical scale; retained as permanent global references.
- **Local pose-reference window:** Full image tokens from the  $k$  most recent frames provide dense visual overlap for accurate registration and relative pose estimation.
- **Trajectory memory:** Older frames are compressed to six lightweight tokens, preserving temporal history to detect drift and maintain global consistency.

**Key takeaway:** GCA combines the anchor context, trajectory memory, and local window into a structured attention mask that retains long-range consistency with bounded per-frame cost.

## Streaming online method

Training loss:

$$\mathcal{L} = \lambda_{\text{depth}} \mathcal{L}_{\text{depth}} + \lambda_{\text{abs-pose}} \mathcal{L}_{\text{abs-pose}} + \lambda_{\text{rel-pose}} \mathcal{L}_{\text{rel-pose}}. \quad (1)$$

The depth loss ( $\mathcal{L}_{\text{depth}}$ ) and absolute pose loss ( $\mathcal{L}_{\text{abs-pose}}$ ) follow the definitions in VGGT [75]:  $\mathcal{L}_{\text{depth}} = \sum_{i=1}^N \left\| \Sigma_i^D \odot (\hat{D}_i - D_i) \right\| + \left\| \Sigma_i^D \odot (\nabla \hat{D}_i - \nabla D_i) \right\| - \alpha \log \Sigma_i^D$ ,  $\mathcal{L}_{\text{abs-pose}} = \sum_{i=1}^N \left\| \hat{\mathbf{P}}_i - \mathbf{P}_i \right\|_{\epsilon}$ . Here,  $\Sigma_i^D$  represents the predicted

Runtime:

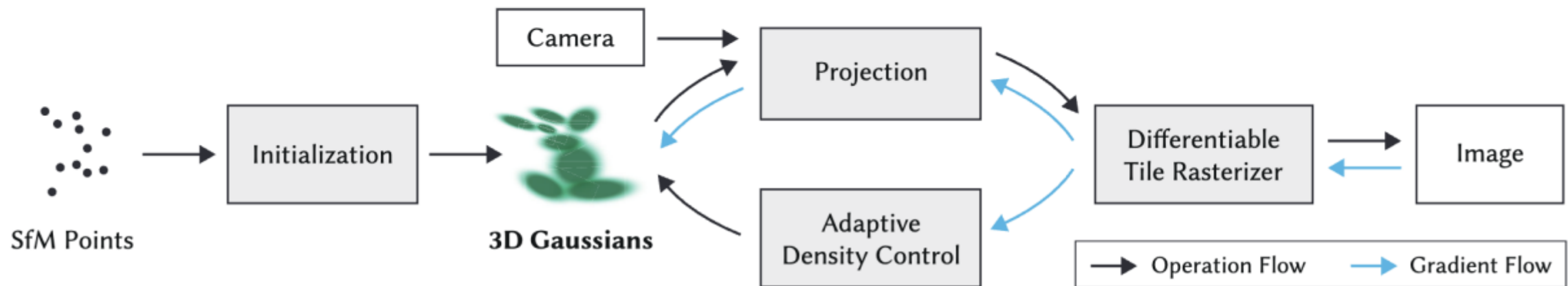
**Table 7. Pose-reference window (size 64) vs. full causal attention.** The bounded window achieves better trajectory accuracy with  $1.7\times$  higher speed and  $2.7\times$  lower memory. The **best** and second best results are marked.

| Window Size | ATE ↓       | RPE-trans ↓ | RPE-rot ↓   | FPS↑         | Mem (GB)↓    |
|-------------|-------------|-------------|-------------|--------------|--------------|
| 64          | <b>5.98</b> | <b>1.33</b> | <u>1.93</u> | <b>20.29</b> | <b>13.28</b> |
| Full        | <u>6.60</u> | <u>1.50</u> | <b>1.71</b> | <u>11.87</u> | <u>36.06</u> |



# 3D Gaussian Splatting

- Splat-based representation
- Use 3D Gaussians instead of points or a mesh.
- It does not include any neural network.



Parameterization of Gaussians:

$$G(\mathbf{x}) = e^{-\frac{1}{2} (\mathbf{x})^T \Sigma^{-1} (\mathbf{x})}$$

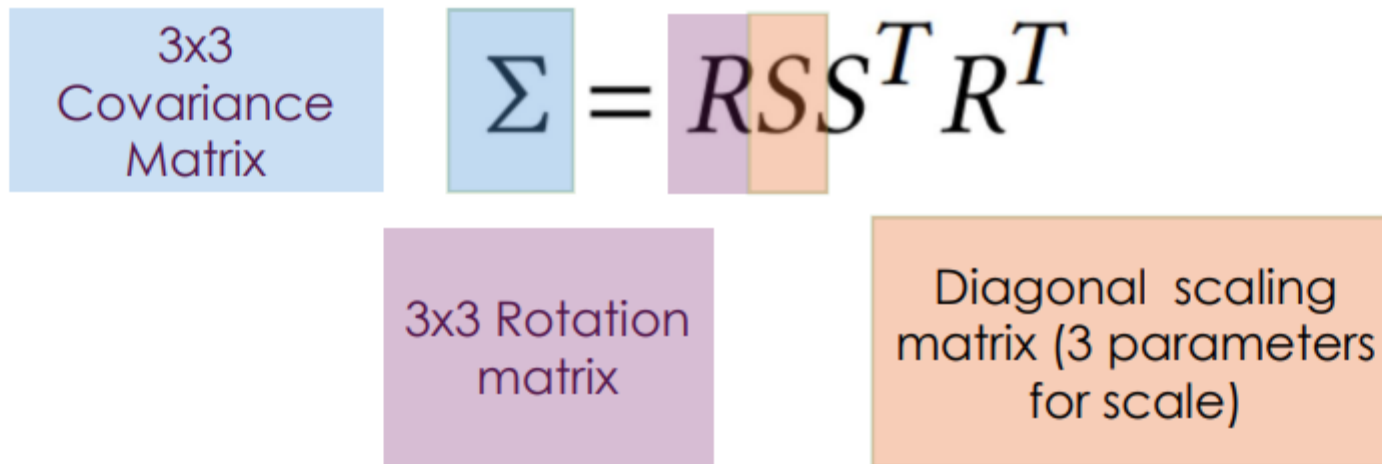
How to  
parametrize 3D  
Gaussian?

3D Gaussian parametrized by:

- 3D point (mean)  $\mu$
- Covariance  $\Sigma$
- Opacity alpha  $\alpha$
- Color  $\mathbf{c}$  – RGB values or spherical harmonics (SH) coefficients.

## How to optimize a covariance matrix $\Sigma$ ?

- Not all symmetric matrices are covariance matrices and gradient updates can easily make them invalid.
- The covariance matrix  $\Sigma$  of a 3D Gaussian is analogous to describing the configuration of an ellipsoid.
- $\Sigma$  has a physical meaning if its a positive-semi definite matrix. So factorize as follows:



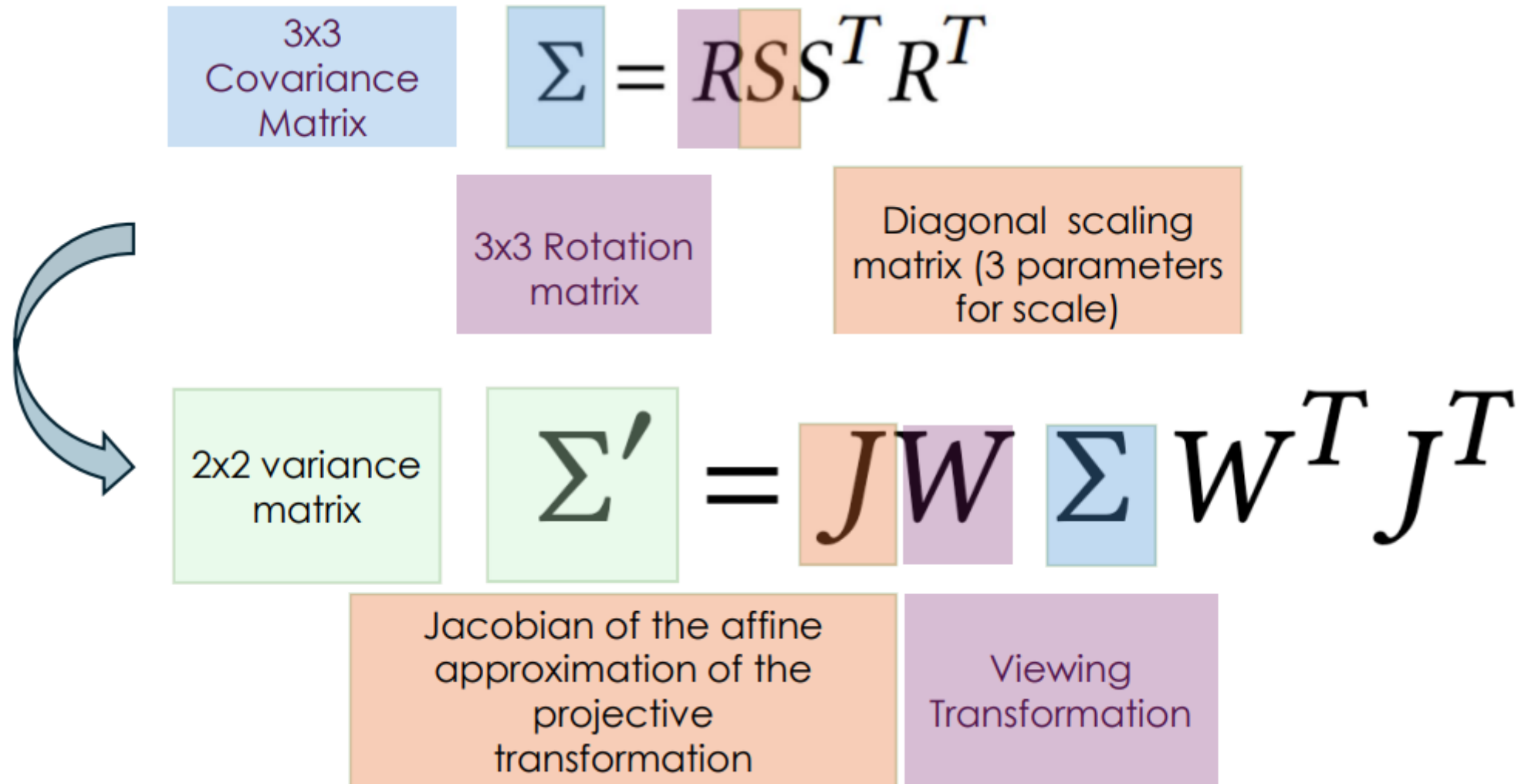
3x3  
Covariance  
Matrix

$$\Sigma = R S S^T R^T$$

3x3 Rotation  
matrix

Diagonal scaling  
matrix (3 parameters  
for scale)

Projection of a covariance matrix  $\Sigma$  into 2D



## Image Formation Model of 3D Gaussian Splatting

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j),$$

Color of a pixel

Color of  
each point

Transmittance



## Optimizing Gaussians:

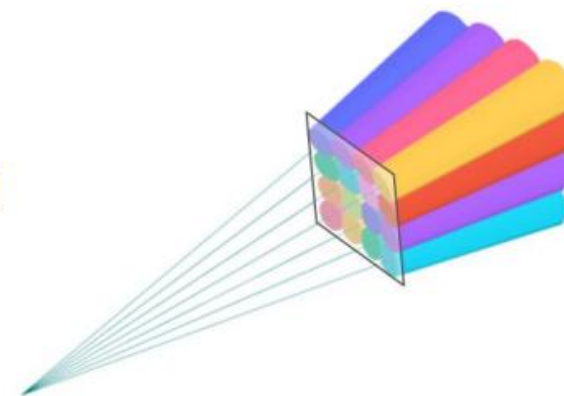
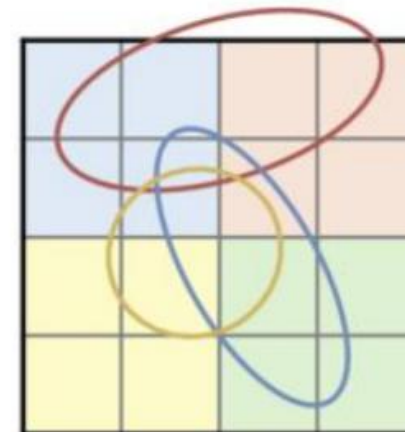
### Optimization

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}$$

How to go from 5 FPS to 100+ FPS?  
(Using the GPU efficiently)

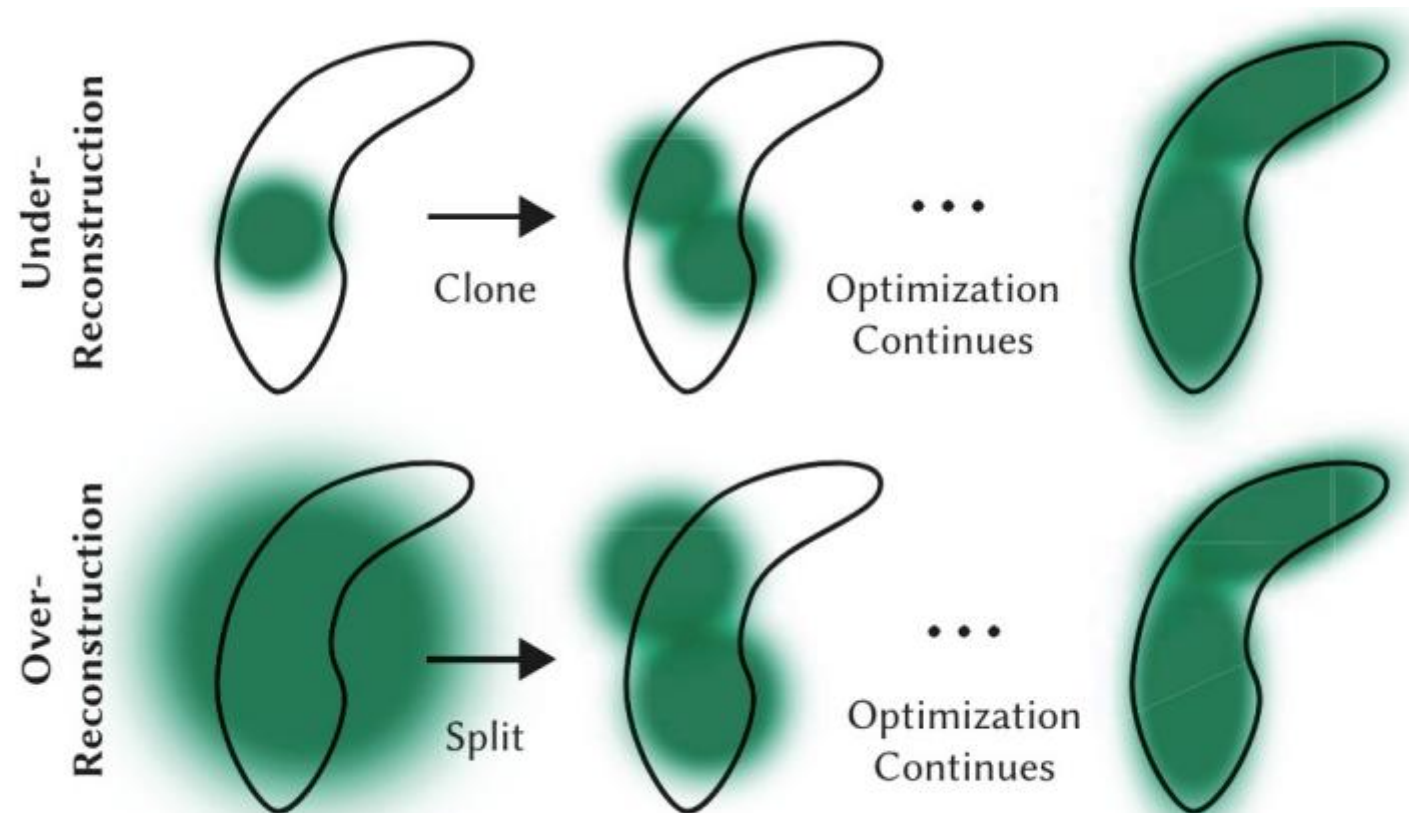
- 1. Tiling
  - Split the image in 16x16 Tiles – helps threads to work coll
- 2. Single global sort
  - GPU sorts millions of primitives fast.

<https://3dgtutorial.github.io/>



# 3D Gaussian Splatting

## Heuristic pruning and spawning operations



# Image formation: NeRF vs Gaussian Splatting

$$C = \sum_{i=1}^N T_i \alpha_i c_i \quad C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j),$$

$$\alpha_i = (1 - \exp(-\sigma_i \delta_i)) \quad \text{and} \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Color of a pixel

Transmittance

Color of  
each point

# NeRF vs Gaussian Splatting

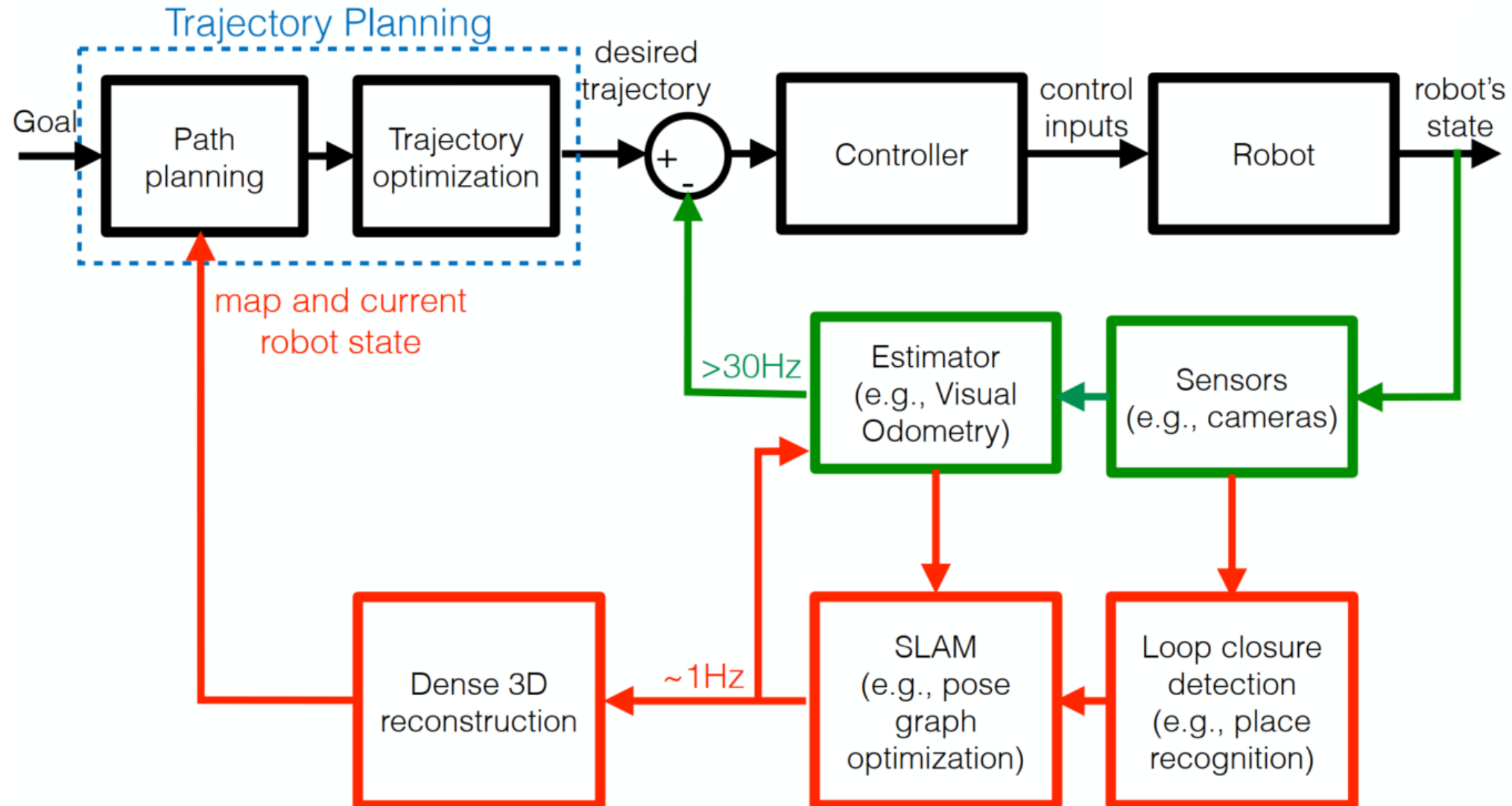
## Core Difference:

- **NeRF** → implicit neural field + **ray marching**
- **Gaussian Splatting** → explicit primitives + **rasterization**

## Why Gaussian Splatting is Faster than NeRF?

- No per-ray MLP evaluation
  - No dense sampling along rays
  - Uses GPU rasterization (splatting)
  - Explicit scene representation
  - Efficient parallel rendering
- 👉 Real-time vs seconds per frame

# SLAM and Dense Mapping in Robotics





1. Awesome Learning-based Odometry, <https://github.com/KwanWaiPang/Awesome-Learning-based-VO-VIO>
2. Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., ... & Leonard, J. J. (2017). Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. IEEE Transactions on robotics, 32(6), 1309-1332.
3. Noah Snavely, CS5670: Computer Vision, Inverse Graphics and Neural Radiance Fields, 2024.
4. Gerard Pons-Moll, Virtual Humans, 3D Gaussian Splatting (3DGS) and Representing Digital Humans with 3DGS, University of Tübingen / MPI-Informatics, 2024.
5. Nikolay Atanasov, Yulun Tian, Sensing & Estimation in Robotics, Dense Mapping, UCSD, 2025.

Any Questions?